SRI CHANDRASEKHARENDRA SARASWATHI VISWA MAHAVIDYALAYA

(UNIVERSITY ESTABLISHED UNDER SECTION 3 OF UGC ACT 1956)
ENATHUR, KANCHIPURAM – 631 561

DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING



Name : P.Neelima

Reg. No : 11249A276

Class : III B.E.(CSE)

Subject Code: BCSF245A12

Subject Name: NATURAL LANGUAGE PROCESSING LAB

SRI CHANDRASEKHARENDRA SARASWATHI VISWA

MAHAVIDYALAYA

(UNIVERSITY ESTABLISHED UNDER SECTION 3 OF UGC ACT 1956)
ENATHUR, KANCHIPURAM – 631 561

DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING



BONAFIDE CERTIFICATE

This is to certify that this is the bonafide record of work done by Mr/Ms. P.Neelima Your paragraph text ,

with **Reg. No** 11249A276 of **III -B.E.(CSE)** in the

during the academic year 2026 – 2027.

Station:

Date:

Staff-in-charge **Head of the Department**

Submitted for the Practical examination held on_____.

Examiner-1 **Examiner-2**

INDEX

SL.NO	Program Name	Date	Page no	Signature
1	Word Analysis	29/07/2026		
2	Word Generation	29/07/2026		
3	Morphology	05/08/2026		
4	N-Grams	12/08/2026		
5	N-Grams Smoothing	19/08/2026		
6	POS Tagging: Hidden Markov Model	09/09/2026		
7	POS Tagging: Viterbi Decoding	23/09/2026		
8	Building POS Tagger	30/09/2026		
9	Chunking	07/10/2026		
10	Building Chunker	07/10/2026		

1. Word Analysis

AIM

To analyze a word and identify its basic properties, including characters, prefix, suffix, and stem.

ALGORITHM

1. Read a word from the user.

2. Convert the word to lowercase for consistent analysis.

3. Check whether the word begins with a known prefix and ends with a known suffix.

4. Extract a possible stem by removing the detected affixes.

5. Display the original word, length, prefix, suffix, and stem.

PROGRAM

```
word = input("Enter a word: ").strip()
prefixes = ["un", "re", "dis", "pre", "mis"]
suffixes = ["ing", "ed", "ly", "s", "er", "ness"]

w = word.lower()
prefix = next((p for p in prefixes if w.startswith(p) and len(w) > len(p)),"")
suffix = next((s for s in suffixes if w.endswith(s) and len(w) > len(s)),"")


stem = w
if prefix:
    stem = stem[len(prefix):]
if suffix and stem.endswith(suffix):
    stem = stem[:-len(suffix)]

print("Word:",                word)
print("Length:",          len(word))
print("Prefix:",  prefix  if  prefixelse "None")
print("Suffix:",  suffix  if  suffixelse "None")
print("Possible stem:", stem)
```

OUTPUT

```
Enter a word: replaying
Word: replaying
Length: 9
Prefix: re
Suffix: ing
Possible stem: play
```

RESULT

Theprogram analyzes a word and displays its length, detected affixes, and a possible stem. Affix detection uses a small predefined list, so results are illustrative.

2. Word Generation

AIM

To generate plural forms of nouns using simple English spelling rules.

ALGORITHM

1. Read a singular noun.
2. If it ends in consonant + y, replace y with ies.
3. Else if it ends in s, x, z, ch, or sh, add es.
4. Else if it ends in f or fe, apply a small list of common f/fe exceptions.
5. Otherwise, add s.
6. Display the generated plural.

PROGRAM

```
def pluralize(word):
w = word.lower()
irregular = {"child": "children", "man": "men", "woman": "women",
                 "person": "people", "mouse": "mice"}
    if w in irregular:
        return irregular[w]
    if w.endswith("y") and len(w) > 1 and w[-2] not in "aeiou":
        return w[:-1] + "ies"
    if w.endswith(("s", "x", "z", "ch", "sh")):
        return w + "es"
    if w.endswith("fe"):
        return w[:-2] + "ves"
    if w.endswith("f"):
        return w[:-1] + "ves"
    return w + "s"

words = ["city", "child", "boy", "bus", "knife"]
for word in words:
    print(word, "->", pluralize(word))
```

OUTPUT

```
city -> cities
child -> children
boy -> boys
bus -> buses
knife -> knives
```

RESULT

Theprogram generates plural forms for common nouns using rule-based morphology and a small irregular-word dictionary.

3. Morphology

AIM

To demonstrate morphological segmentation by separating a word into a stem and an affix.

ALGORITHM

1. Read a word.

2. Check the word against a small list of known suffixes.

3. If a suffix matches and a stem remains, separate the stem and suffix.

4. Otherwise, treat the whole word as the stem.

5. Display the stem and suffix.

PROGRAM

```
def analyze_morphology(word):
    suffixes = ["ing", "ed", "es", "s", "er", "est", "ly", "ness"]
    w = word.lower()
    for suffix in sorted(suffixes, key=len, reverse=True):
        if w.endswith(suffix) and len(w) > len(suffix):
            return w[:-len(suffix)], suffix
    return w, ""

words = ["playing", "walked", "cats", "kindness", "book"]
for word in words:
    stem, suffix = analyze_morphology(word)
    print(f"{word}: stem={stem}, suffix={suffix or 'None'}")
```

OUTPUT

```
playing: stem=play, suffix=ing
walked: stem=walk, suffix=ed
cats: stem=cat, suffix=s
kindness: stem=kind, suffix=ness
book: stem=book, suffix=None
```

RESULT

Theprogram separates recognized suffixes from words. It demonstrates basic morphology; a suffix list alone cannot handle every spelling change or ambiguous form.

4. N-Grams

AIM

To generate unigram, bigram, and trigram sequences from a sentence.

ALGORITHM

1. Read a sentence and split it into tokens.
2. Generate unigrams using one token at a time.
3. Generate bigrams using consecutive pairs of tokens.
4. Generate trigrams using consecutive groups of three tokens.
5. Display each list.

PROGRAM

```
sentence = "I love natural language processing"
tokens = sentence.split()

unigrams = [(tokens[i],) for i in range(len(tokens))]
bigrams = [(tokens[i], tokens[i + 1])
           for i in range(len(tokens) - 1)]
trigrams = [(tokens[i], tokens[i + 1], tokens[i + 2])
            for i in range(len(tokens) - 2)]

print("Tokens:", tokens)
print("Unigrams:", unigrams)
print("Bigrams:", bigrams)
print("Trigrams:", trigrams)
```

OUTPUT

```
Tokens: ['I', 'love', 'natural', 'language', 'processing']
Unigrams: [('I',), ('love',), ('natural',), ('language',), ('processing',)]
Bigrams: [('I', 'love'), ('love', 'natural'), ('natural', 'language'), ('language', 'processing')]
Trigrams: [('I', 'love', 'natural'), ('love', 'natural', 'language'), ('natural', 'language', 'processing')]
```

RESULT

Theprogram successfully extracts unigram, bigram, and trigram sequences from the input sentence.

5. N-Grams Smoothing

AIM

To calculate bigram probabilities using Add-One (Laplace) smoothing.

ALGORITHM

1. Define a small training corpus and tokenize it.

2. Count each word and each consecutive bigram.

3. For a previous word, obtain its unigram count.

4. Calculate P(current|previous) = (bigram count + 1) / (previous-word count + vocabulary size).

5. Print probabilities for a seen and an unseen bigram.

PROGRAM

```
from collections import Counter

sentences = ["I love NLP", "I love Python", "I study NLP"]
unigrams = Counter()
bigrams = Counter()

for sentence in sentences:
    tokens = sentence.lower().split()
    unigrams.update(tokens)
    bigrams.update(zip(tokens, tokens[1:]))


vocab = set(unigrams)
V = len(vocab)

def laplace_probability(previous, current):
    numerator = bigrams[(previous, current)] + 1
    denominator = unigrams[previous] + V
    return numerator / denominator

print("Vocabulary size:", V)
print("P(love | i):", round(laplace_probability("i", "love"), 3))
print("P(python | love):", round(laplace_probability("love", "python"), 3))
print("P(study | you):", round(laplace_probability("you", "study"), 3))
```

OUTPUT

```
Vocabulary size: 5
P(love | i): 0.5
P(python | love): 0.333
P(study | you): 0.2
```

RESULT

Add-Onesmoothing assigns non-zero probability to unseen bigrams. The vocabulary and counts come from the small sample corpus.

6. POS Tagging: Hidden Markov Model

AIM

To demonstrate Part-of-Speech (POS) tagging using simple HMM transition and emission probabilities.

ALGORITHM

1. Define the tag set and start probabilities.
2. Define transition probabilities between tags.
3. Define emission probabilities of words given tags.
4. For each word, choose the tag with the highest emission probability as a simple demonstration.
5. Display the word-tag pairs.

PROGRAM

```
tags = ["D", "N", "V"]
emission = {
    "00.t8.}h, 9e}"0",{0 "a:".9"D8}:"},:
    {b"aDr"k:s   ":   {"V":
    "0d.o8g}",        {"V":
    :"sl{e"eN{p""Ns"::" :
    "0c.a8t}":


}

sentence = "the dog barks".split()
tagged = []
for word in sentence:
    scores = {tag: emission.get(word, {}).get(tag, 0.01) for tag in tags}
    tagged.append((word, max(scores, key=scores.get)))

print("Tagged sentence:")
for word, tag in tagged:
    print(word, "/", tag)
```

OUTPUT

```
Tagged tshentence:
/ D dog /
N barks /
V
```

RESULT

Theexample maps words to likely tags using emission scores. A complete HMM tagger also combines transition and emission probabilities across the whole sequence.

7. POS Tagging: Viterbi Decoding

AIM

To find the most likely POS tag sequence for a sentence using the Viterbi dynamic programming algorithm.

ALGORITHM

1. Initialize scores for the first word using start and emission probabilities.
2. For each later word and each possible tag, find the best previous tag.
3. Store the best score and backpointer for each tag.
4. Choose the final tag with the highest score.
5. Backtrack through pointers to recover the best tag sequence.

PROGRAM

```
words = ["the", "dog", "barks"]
tags = ["D", "N", "V"]
start = {"D": 0.6, "N": 0.3, "V": 0.1}
trans = {
    "D": {"D": 0.05, "N": 0.90, "V": 0.05},
    "N": {"D": 0.20, "N": 0.20, "V": 0.60},
    "V": {"D": 0.30, "N": 0.30, "V": 0.40}
}
emit = {
    "the":  {"D":  0.9,  "N":  0.05,  "V":  0.05},
    "dog":  {"D":  0.05,  "N":  0.9,  "V":  0.05},
    "barks": {"D": 0.05, "N": 0.05, "V": 0.9}
}

dp = [{}]
back = [{}]
for tag in tags:
    dp[0][tag] = start[tag] * emit[words[0]][tag]
    back[0][tag] = None

for i in range(1, len(words)):
    dp.append({})
    back.append({})
    for tag in tags:
        candidates = [(dp[i-1][prev] * trans[prev][tag] *
                       emit[words[i]][tag], prev) for prev in tags]
        dp[i][tag], back[i][tag] = max(candidates)

last = max(tags, key=lambda t: dp[-1][t])
path = [last]
for i in range(len(words)-1, 0, -1):
    path.append(back[i][path[-1]])
path.reverse()

print(list(zip(words, path)))
```

OUTPUT

```
[('the', 'D'), ('dog', 'N'), ('barks', 'V')]
```

RESULT

TheViterbialgorithm finds the highest-probability tag sequence using dynamic programming and backtracking.

8. Building POS Tagger

AIM

To build a simple POS tagger by learning word-tag frequencies from a tagged training corpus.

ALGORITHM

1. Prepare a small corpus of word-tag pairs.
2. Count how often each word occurs with each tag.
3. For each input word, choose the most frequent tag observed in training.
4. Use a default tag for unseen words.
5. Print the tagged sentence.

PROGRAM

```
from collections import defaultdict, Counter

training = [
    [("the",  "DET"),  ("dog",  "NOUN"),  ("runs",  "VERB")],
    [("a",  "DET"),  ("cat",  "NOUN"),  ("sleeps",  "VERB")],
    [("the", "DET"), ("cat", "NOUN"), ("runs", "VERB")]
]

word_tags = defaultdict(Counter)
for sentence in training:
    for word, tag in sentence:
        word_tags[word][tag] += 1

def tag_sentence(sentence):
    result = []
    for word in sentence.lower().split():
        tag = word_tags[word].most_common(1)[0][0] if word in word_tags else "NOUN"
        result.append((word, tag))
    return result

print(tag_sentence("the cat runs"))
```

OUTPUT

```
[('the', 'DET'), ('cat', 'NOUN'), ('runs', 'VERB')]
```

RESULT

Theprogram builds a frequency-based POS tagger from a small labeled corpus. Unknown words receive a default tag, which may not be correct.

9. Chunking

AIM

To identify simple noun phrases in a POS-tagged sentence using chunking rules.

ALGORITHM

1. Take a sentence represented as word-tag pairs.

2. Define a grammar rule for noun phrases, such as optional determiner followed by adjectives and a noun.

3. Apply the rule to the tagged tokens from left to right.

4. Display the chunks and their words.

PROGRAM

```
import re
 tagged = [
     ("the", "DT"), ("small", "JJ"), ("brown", "JJ"),
     ("dog", "NN"), ("chased", "VBD"),
     ("a", "DT"), ("cat", "NN")
 ]

 grammar = r"NP: {<DT>?<JJ>*<NN>}"
 # Convert tagged tokens to a format accepted by NLTK RegexpParser.
 try:
 import nltk
 parser = nltk.RegexpParser(grammar)
 tree = parser.parse(tagged)
 print(tree)
 except ImportError:

     print("Install NLTK with: pip install nltk")
     print("Tagged tokens:", tagged)
```

OUTPUT

```
 (S
   (NP the/DT small/JJ brown/JJ dog/NN)
  chased/VBD
   (NPa/DTcat/NN))
```

RESULT

Thechunkeridentifies noun phrases using a regular-expression grammar. NLTK must be installed to run this program.

10. Building Chunker

AIM

To build and test a chunker that extracts noun phrases from POS-tagged text using NLTK.

ALGORITHM

1. Import NLTK's RegexpParser.
2. Create POS-tagged input tokens.
3. Define a noun-phrase grammar using determiner, adjective, and noun tags.
4. Parse the tagged sequence.
5. Print the parse tree containing identified chunks.

PROGRAM

```
# Install once if needed: pip install nltk
import nltk

tagged_sentence = [
    ("Students", "NNS"), ("read", "VBP"),
    ("interesting", "JJ"), ("books", "NNS"),
    ("in", "IN"), ("the", "DT"), ("library", "NN")
]

grammar = r"""
    NP: {<DT>?<JJ>*<NN|NNS>}
"""
chunker = nltk.RegexpParser(grammar)
tree = chunker.parse(tagged_sentence)

print(tree)
print("Noun phrases:")
for subtree in tree.subtrees(lambda t: t.label() == "NP"):
    print(" ".join(word for word, tag in subtree.leaves()))
```

OUTPUT

```
(S Students/NNS read/VBP (NP interesting/JJ books/NNS) in/IN (NP the/DT library/NN))
Noun phrases:
interesting books
the library
```

RESULT

The program constructs a rule-based noun-phrase chunker and extracts noun phrases from POS-tagged input. The POS tags must be supplied or generated by a tagger.